



PDF Download
3631461.3631956.pdf
22 January 2026
Total Citations: 5
Total Downloads: 253

Latest updates: <https://dl.acm.org/doi/10.1145/3631461.3631956>

RESEARCH-ARTICLE

Efficient Air Quality Index Prediction on Resource-Constrained Devices using TinyML: Design, Implementation, and Evaluation

ARKO DATTA, National Institute of Technology, Durgapur, Durgapur, WB, India

ANIRUDDHA PAL, Bengal College of Engineering & Technology, Durgapur, WB, India

RANBIR MARANDI, National Institute of Technology, Durgapur, Durgapur, WB, India

NILANJAN CHATTARAJ, National Institute of Technology, Durgapur, Durgapur, WB, India

SUBRATA K NANDI, National Institute of Technology, Durgapur, Durgapur, WB, India

SUJOY SAHA, National Institute of Technology, Durgapur, Durgapur, WB, India

Open Access Support provided by:

National Institute of Technology, Durgapur

Bengal College of Engineering & Technology

Published: 04 January 2024

[Citation in BibTeX format](#)

ICDCN '24: International Conference on
Distributed Computing and Networking
January 4 - 7, 2024
Chennai, India

Efficient Air Quality Index Prediction on Resource-Constrained Devices using TinyML: Design, Implementation, and Evaluation

Arko Datta*
contact.arkodatta@gmail.com
NIT Durgapur
India

Aniruddha Pal*
aniruddhapal211316@gmail.com
BCET
India

Ranbir Marandi
marandiranbir12@gmail.com
NIT Durgapur
India

Nilanjan Chattaraj
nilanjan.chattaraj@gmail.com
NIT Durgapur
India

Subrata Nandi
subrata.nandi@gmail.com
NIT Durgapur
India

Sujoy Saha
sujoy.ju@gmail.com
NIT Durgapur
India

ABSTRACT

These days, mobile/handheld air quality monitoring devices primarily sense pollutants and send them to the cloud without performing intelligent processing, such as soft calibration, sensor fault detection, or any onboard predictions. This paper explores the feasibility of harnessing Tiny Machine Learning (TinyML) to develop a compact system for inferring environmental pollution levels at a device's location. Our primary challenge lies in fitting a trained machine-learning model within the device's limited memory without compromising its accuracy or inference speed. Building upon the correlation between the Air Quality Index (AQI) and various meteorological factors, we propose a TinyML-based framework that predicts AQI using temperature, humidity, internet-sourced meteorological data, and temporal information collected through a web crawler. Our contributions encompass sensor activation, system design on a Raspberry Pi Pico W board, and hyperparameter tuning of machine learning models, culminating in an efficient XGBoost implementation within the 2MB memory constraint, achieving 75.2% accuracy with a 1615 μ s latency. Moreover, an accuracy of 60% was observed when a real-time validation was carried out. As per our knowledge, this system is the first-of-its-kind device that tries to embed some intelligence into a mobile air quality sensing device.

CCS CONCEPTS

• **Computing methodologies** → **Classification and regression trees**; • **Human-centered computing** → **Ubiquitous and mobile computing systems and tools**.

KEYWORDS

TinyML, Performance, Pollution Monitoring, Latency, System Prototype

*Both authors contributed equally to this research.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ICDCN '24, January 04–07, 2024, Chennai, India

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1673-7/24/01.
<https://doi.org/10.1145/3631461.3631956>

ACM Reference Format:

Arko Datta, Aniruddha Pal, Ranbir Marandi, Nilanjan Chattaraj, Subrata Nandi, and Sujoy Saha. 2024. Efficient Air Quality Index Prediction on Resource-Constrained Devices using TinyML: Design, Implementation, and Evaluation. In *25th International Conference on Distributed Computing and Networking (ICDCN '24)*, January 04–07, 2024, Chennai, India. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3631461.3631956>

1 INTRODUCTION

The quality of air is degrading day by day due to increased urbanisation. Vehicular emissions, industrial emissions, smoke from cigarettes and various other factors have poisoned the air we breathe, raising the Air Quality Index of our surroundings. Even though cities have Air Quality Monitoring Stations, those stations are sparsely populated. We need to know the AQI of the air we breathe; that is, the AQI of the air in our vicinity. Hence, there is a need for such a system that will provide a personalised Air Quality Index based on our exact location.

TinyML boards[4] are now being manufactured with Machine Learning cores embedded in them. Instead of sending the collected data to a cloud service for some inference using machine learning models, the machine learning models can now be applied on board. Several boards currently support TinyML, for example, Apollo3, Raspberry Pi Pico W, and Arduino Nano RP2040 Connect. These boards support development in various programming languages, but C/C++ and Micropython are more widely used. TinyML is gaining popularity in the solutions based on IoT and embedded systems, particularly in healthcare, where it ensures patient data privacy and aids COVID-19 monitoring[6]. It is used in wearable vital sign monitors, hearing aids, personalised home healthcare, industrial automation, and monitors an industrial asset's physical conditions [2]. TinyML-based systems automate smart agriculture [7] and monitor vehicular emissions [1], making it a versatile technology for real-time, edge-based data analysis and decision-making.

In an earlier research work, we discovered a high correlation between the AQI of an area and the temperature, humidity and other meteorological factors like rain, cloud cover, and so on. We also found a correlation between the AQI and the season and time of the day. Hence, we proposed a framework [5][3], which uses these factors to predict the AQI of a place on a personal scale. However, even though the sensing of humidity and temperature was being done on board, the machine learning model was executed on the cloud.

Understanding these opportunities from existing literature, we define the primary goal of our paper as follows: (a) *Is it possible to utilise the on-board machine learning capabilities of a TinyML board, which has limited computational resources (including a low clock speed, 264 KB of SRAM, and 2 MB of Flash memory), and is equipped with both a humidity and temperature sensor, as well as the ability to retrieve meteorological data via on-board web crawling using WiFi, to determine the Air Quality Index (AQI) for a specific location?* (b) *In this environmental monitoring system, is it possible to achieve real-time verification of the inferred AQI and real-time AQI data?*

Our contributions towards the development of this system are:

(a) Activation of Sensors: Some of the sensors, like DHT22 and PMSA003 had either none or erroneous device driver implementation in C/C++. We have developed the device drivers for these devices to mitigate such cases.

(b) System Design and Development: We have used Raspberry Pi Pico W, which is a TinyML-enabled board, and low-cost sensors to develop the system to gather the temperature, humidity and dust particle data. ML models are deployed into the board to predict the AQI based on Temperature, Humidity, meteorological and temporal data. To verify the predicted AQI with the real AQI, PMSA003 dust particle sensor is being utilised. An SD card is available for storage and an OLED screen is provided for display.

(c) Hyperparameter Tuning and Selection of Machine Learning Models: We have utilised three machine learning models: Decision Tree, Random Forest and XGBoost. The hyperparameters of each model have been tuned so that the models, when ported to raw C/C++ or raw Micropython, are within the 2MB flash memory of our TinyML board while maintaining the highest accuracy possible. We have found that the most accurate model that fits in our TinyML board is the XGBoost implementation in C/C++ (around 75.2%), with a size of 3656.485 kB and a latency of 1615 μ s.

The paper is divided into six sections. Section 3 discusses the data acquisition and feature engineering process for the system. Section 4 discusses the machine learning model we have used. Section 5 delves into the results obtained and insights gathered while developing the system. Finally, Section 6 concludes the discussion and speculates the future scope of the system.

2 SYSTEM ARCHITECTURE

In our system development, we have employed various sensors, including humidity and temperature sensors and a Dust particle sensor, for efficient Environmental monitoring. All these peripherals are seamlessly connected to a TinyML board like Raspberry Pi Pico W. An SD card module is utilized for data storage. Meanwhile, the sensing data output and model results are displayed on an OLED screen.

In the domain of TinyML, real-time sensor data is fed into a machine learning or deep learning model deployed on a low-powered microcontroller, enabling it to perform intelligent tasks. We have opted to execute the models in raw C/C++ and raw Micropython for our project. To successfully interface all our sensors with the C/C++ models, we require C/C++ implementations specifically tailored for our sensors. Although most sensors have existing Arduino implementations, locating suitable C/C++ implementations poses a

challenge. Thus, we focus on obtaining or developing the necessary C/C++ implementations to integrate the sensors with our models seamlessly.

Humidity and Temperature Sensor: The DHT22, also known as the AM2302 sensor, is a digital temperature and humidity sensor utilizing a capacitive humidity sensor and a thermistor to measure environmental conditions. With a micro-controller, this sensor communicates via GPIO pins, including VCC, GND, and Data. This sensor will provide the temperature and humidity of the surroundings to our Model.

The DHT22, C/C++ implementation was available in `pico_examples` github repository¹, but when this implementation was used, the temperature and humidity data remained at a constant, random value. This was because the data retrieval timing in DHT22 is very sensitive, and the Raspberry Pi Pico W board seems to be off by some value. Hence, we added an 18ms delay in the initial handshake code of DHT22². According to the datasheet, the DHT22 sensor sends a 26-28 μ s pulse for bit value 0 and 70 μ s for bit value 1. However, upon analyzing the pulse timings using a logical analyzer attached to a DHT22, it was found that any pulse greater than 50 μ s should mean bit 1. In the code provided in the repository, the `sleep_ms()` function was used, which has some alarm-related overhead. When this function is used, the loop is delayed by 1.3ms, and the count variable increases too slowly. This is why the original code had a threshold `if(count>16)`. So, instead of using `sleep_ms()` function we can use `time_us_32()` or `busy_wait_us_32()` function and use a threshold of 50 that is `if(count>50)`³.

Dust Particle Sensor: The PMSA003 sensor is a digital particulate matter (PM) sensor that employs a laser to detect and measure the concentration of PM1.0, PM2.5 and PM10 particles in the air. The sensor has specific pins that employ the UART protocol for communication. The PMSA003 consists of 32 8-bit registers. The first two registers are start characters, with their data fixed at 0x42 and 0x4d, respectively. The following two registers contain the frame length. Registers 0x04 to 0x1d contain the data bytes, with registers 0x1c and 0x1d containing the version and error codes, respectively. Registers 0x1e and 0x1f contain the check codes. The data is read byte by byte to infer the PM1.0, PM2.5, and PM10.0 data, which are stored in the registers. The implementation of PMSA003 was made available publicly by the manufacturer in micropython. We then took reference from this Micropython implementation and the datasheet⁴ of PMSA003 and wrote its implementation in C/C++ as our model interfaces in C/C++.

Storage: The output of the readings from our system is stored in an SD Card. This SD card adapter works using the SPI physical layer protocol. Due to the constrained 2 MB flash memory of Raspberry Pi Pico, we are using an SD Card for storing the temperature and humidity data from the DHT22 Sensor, PM1.0, PM2.5, and PM10 data from PMSA003, the rest of the features crawled from the web, the Real AQI and the Model AQI. Furthermore, this data can be used to retrain or finetune the model.

¹<https://github.com/raspberrypi/pico-examples>

²This was suggested by <https://github.com/raspberrypi/pico-examples/issues/11#issuecomment-768467615>

³This was suggested by: <https://github.com/raspberrypi/pico-examples/issues/11#issuecomment-912619076>

⁴https://m.eleparts.co.kr/data/goods_old/data/PMSA003.pdf

Crawler: We have used TLS Client as a web crawler for crawling features other than temperature and humidity from publicly available web APIs. We have used `api.openweathermap.org` API, which is a public weather API for crawling Feels like, Hourly Minimum Temperature, Hourly Maximum Temperature, Pressure, Wind Speed, Wind Direction, Average Hourly Precipitation, Percentage Cloud Cover and Weather and also used `www.timeapi.io` which is a public time API for crawling Hour of Day, Activity-based cluster, Season, Month, Day of Week.

Display: The system displays the Real AQI and the predicted AQI on a 0.91" OLED display. The display is connected via I2C Serial.

Table 1 shows the issues faced while interfacing the sensors and our approach to solving them. The issues were mostly faced in the C/C++ implementation of drivers.

Sensor Name	Problem Faced	Solution Approach
Humidity Temperature Sensor (DHT22)	Upon using the C/C++ driver implementation available in the <code>pico_examples</code> github repository, the temperature and humidity data remained at a constant, random value.	Instead of using <code>sleep_ms()</code> function, <code>time_us 32()</code> or <code>busy_wait_us 32()</code> function and a threshold of 50 can be used.
Dust Particle Sensor (PMSA003)	The C/C++ driver implementation are unavailable as per our knowledge.	Reference from the micropython implementation and the official datasheet of PMSA003 were taken, and the C/C++ implementation was written.

Table 1: Issues faced while interfacing sensors and solution approach used.

System Prototype: Our proposed prototype and its circuit diagram are as shown in Figure 1. The Raspberry Pi Pico board is attached to a Pico Air Monitoring Expansion, an open-source portable sensor developed by SB components. It has an embedded dust particle sensor (PMSA003) and an on-board 0.91" OLED display (SSD1306). The GPIO pins GP0 and GP1 pins of the Raspberry Pi Pico W board are used to connect the dust particle sensors' PMS RX and PMS TX pins, respectively. The GPIO pins GP21 and GP22 are connected to the OLED screen's SCL and SDA pins, respectively. External breakout pins are available on the board as well in order to accommodate external sensors and peripherals. We have connected a DHT22 Temperature and Humidity sensor with the system to gather real-time humidity and temperature data. An SD card is also used in the system so that we can store the raw data and the results in it. A Li-Po battery pack is used to power the entire system. We have ensured that each component should have a common ground. A Micro-USB charging cable is also added to the system.

Additionally, a switch has been provided to turn the current flow on or off. The cost of building this prototype has been shown in Table 2.

Component Name	Approximate Cost (in USD)
Raspberry Pi Pico W	\$9.61
DHT22 Humidity and Temperature Sensor	\$12.01
PMSA003 Dust Particle Sensor (Pico Air Monitoring Expansion Board)	\$60.03
SD Card and Adapter	\$9.61
Li-Po Battery	\$6.01
Prototype overhead	\$6.01
TOTAL	\$103.28

Table 2: Approximate cost of building the prototype

The raw data of temperature, humidity and PM1.0, PM2.5, and PM10 are retrieved from the DHT22 and PMSA003 sensors of the system. Then, we use the crawler to crawl the rest of the remaining meteorological features and the current local time, month and day of the week from the publicly available web APIs. The hour of the day, activity-based cluster and season are then calculated from the time provided.

We trained three models for the machine learning model: Decision Tree, Random Forest and XGBoost. The machine learning model, after being trained, was saved using the pickle library in Python. The pickled machine learning model was converted into raw C/C++ or Micropython codes using the `m2cgen` library. Unfortunately, for the XGBoost model, `m2cgen` was causing issues while generating the raw C code. Hence, we have used the `micromlgen` library to implement XGBoost in C/C++. The model's hyperparameters should be chosen particularly for the constraint of Flash Memory in Raspberry Pi Pico W and also considering the usage of the model as we usually might face the Bytecode Overflow Error. The ByteCode overflow Error usually occurs in Micropython when the executed Micropython code requires more SRAM than available on the Raspberry Pi Pico because the Micropython interpreter has to convert each Micropython line of code into Machine Language which requires RAM. As the number of lines in Micropython code increases, the amount of RAM required also increases.

3 DATA ACQUISITION & FEATURE ENGINEERING

To train our models, we acquired the data from Air Quality Monitoring Devices, which were used while developing a previous framework[5][3]. Sixteen features were selected to predict the AQI, among which Humidity and Temperature had very high correlations. Along with that, several meteorological and temporal features were selected.

3.1 Data Acquisition

In our proposed air quality prediction framework[5][3], we gathered several environmental data using low-cost sensors and the

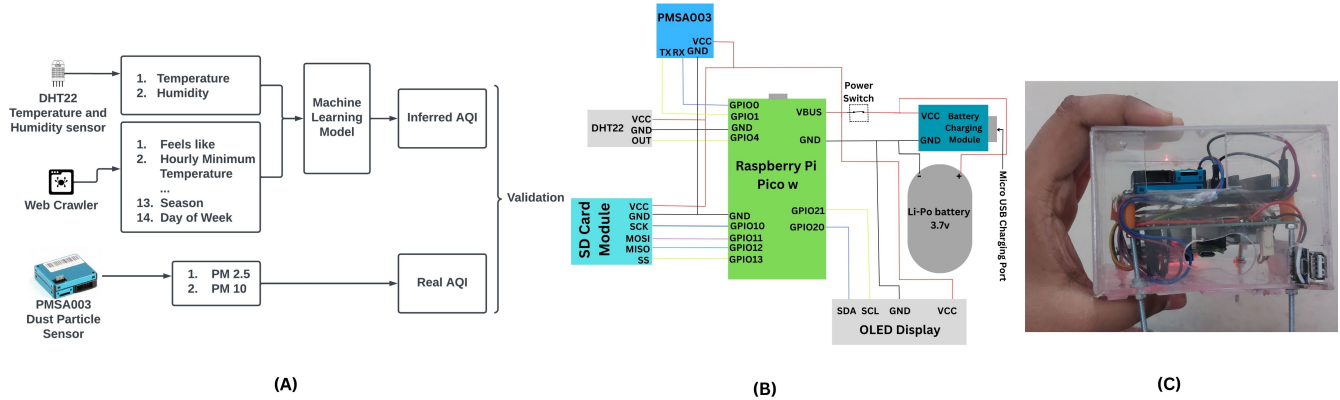


Figure 1: (A) The logical schematics of the prototype. (B): The circuit diagram of the prototype. The VBus of the Raspberry Pi Pico W module is connected to a 3.7V Li-Po battery via a power switch. A charging module is provided as well. The Pico Monitoring Expansion connects the RX pin of the PMSA003 module to GP0 of the Raspberry Pi Pico W, and the TX of the PMSA003 is connected to GP1 of the Raspberry Pi Pico W. The Air Monitoring Expansion also connects the SDA of the OLED screen to GP20 and the SCL of the OLED screen to GP21. The output of the DHT22 is connected to GP4. For the SD card module, SCL is connected to GP10, MOSI is connected to GP11, MISO is connected to GP12, and SS is connected to GP13. The Vin/Vcc and ground of every component are connected to the Vout and ground of the Li-Po battery. (C): The image of the prototype.

corresponding meteorological and temporal data by crawling the web and calculated the corresponding AQI. In that framework, we had used Air Quality Monitoring Devices (AQMDs). These AQMDs were deployed in four strategic locations in our target city (Durgapur). We had gathered data for seventeen months via the AQMDs.

3.2 Feature Engineering

We analyzed that temperature and humidity analysis shows strong correlations with AQI classes, with lower humidity and higher temperature indicating better air quality. Moreover, city-specific climate patterns affect AQI parameter impacts.

Spatial features affect AQI distribution, but city-wide behaviour remains consistent. Temporal analysis over 12 and 17 months reveals varying AQI patterns across time, with peak pollution during mornings and evenings. Hence, to train our model, we use sixteen features as inputs for our AQI model, which are - f_1 : Temperature, f_2 : Humidity, f_3 : Feels like, or the apparent temperature, f_4 : Hourly Minimum Temperature, f_5 : Hourly Maximum Temperature, f_6 : Pressure, f_7 : Wind Speed, f_8 : Wind Direction, f_9 : Average Hourly Precipitation, which is the amount of rainfall in the current hour, f_{10} : Percentage Cloud Cover, f_{11} : Weather Type like cloudy, sunny, hazy, overcast, thunderstorm etc., f_{12} : Hour of Day, f_{13} : Activity-based cluster, which are determined based on the hour of the day, since human activity varies with time of the day, f_{14} : Month, f_{15} : Season such as Winter, Summer, Monsoon, and f_{16} : Day of Week. Temperature and humidity are real-time data; the rest are obtained from open-source APIs.

⁵The data set is open and available at <https://github.com/prasenjit52282/AQuaMoHo/tree/master/Data>

4 PROPOSED MODEL & PARAMETER TUNING

We have embedded a machine learning model on the micro-controller unit to process the humidity, temperature and other meteorological and temporal features and predict the AQI. We have trained Decision Trees, Random Forests and XGBoost machine learning models for our system and compared their size and performance to select the best among them.

4.1 Methodology

Machine learning models in Scikit Learn library in Python were created for Decision Trees, Random Forests and XGBoost. A provision was added to save the machine learning models generated as .sav files, using the pickle library. Grid-search approach was used to iterate over various hyperparameters, and the output models were stored. Finally, the most accurate model that fits in the size constraint of our micro-controller unit was used.

4.2 Models Used

We trained models for three machine learning algorithms: Decision Trees, Random Forests and XGBoost. In Decision Trees, we create a model that predicts the value of a target variable by learning simple decision rules inferred from the data features. However, Decision Trees suffer from the problem of overfitting. To mitigate the problem of overfitting, we use the Random Forest algorithm, which clubs the trees together. While predicting the class, this algorithm polls the classification from every tree and returns the type with the maximum votes. Meanwhile, XGBoost is a scalable, distributed gradient-boosted decision tree-based algorithm that uses an ensemble of weak prediction models, or decision trees, to return a firm prediction. In XGBoost, the decision trees are built parallelly instead of sequentially.

4.3 Hyperparameter Tuning

To get a more accurate classification model, hyperparameter tuning is a useful process. Different Machine Learning models contain different hyperparameters. In our case, we have adjusted a couple of hyperparameters to maximize our accuracy.

Model Name	Minimum Samples per leaf (min_samples_leaf)	Maximum Depth (max_depth)	Test Accuracy
Decision Tree	1	19	0.6876072041
	3	11	0.657161235
	4	1	0.4618353345

Model Name	Number of Estimators (n_est)	Maximum Depth (max_depth)	Test Accuracy
Random Forest	18	18	0.7450686106
	4	10	0.6633790738
	4	2	0.5

Model Name	Number of Estimators (n_estimators)	Learning Rate (learning_rate)	Maximum Depth (max_depth)	Test Accuracy
XGBoost	20	0.7	20	0.7596483705
	12	0.01	18	0.7101200686
	6	0.2	2	0.5336620926

Table 3: Hyperparameter tuning of models: Decision Tree, Random Forest and XGBoost

From Table 3, we can see that the most accurate classification is done by XGBoost (around 76%). Since all the above algorithms are tree-based, we can also see that the accuracy depends on the number of estimators (in the case of Random Forest and XGBoost) and the maximum depth.

5 RESULT ANALYSIS

In our proposed system, machine learning algorithms are being fit into a micro-controller unit with minimal computational capabilities. To accommodate this, the models are converted into simple if-else codes written in C/C++ or Micropython using the libraries `m2cgen` and `micromlgen`.

We aim to find the most accurate model that will fit the size constraints of the micro-controller and the SDK being used. While building the firmware of our device, if any memory constraint is violated, the firmware will not be built. Moreover, if there is too much complexity in our code, a firmware being built using micropython will throw a Bytecode Overflow error.

Even though we are able to achieve around 76% accuracy on the machine in which the model is being trained, we still need to select hyperparameters that generate a model that occupies lesser space because of the space constraints of the micro-controller unit.

Comparison Among Machine Learning Models: As observed in Figure 2(C), XGBoost is the most accurate classifier that could

be fitted in our TinyML board, followed by Random Forest, and Decision Tree is the least accurate. The high accuracy of XGBoost is because the XGBoost is a Boosting technique. This means that the model predicts something initially. Then, it self-analyses its mistake as a predictive toiler and gives more weightage to the data points in which it made a wrong prediction in the next iteration. In contrast, a Random forest is just a collection of trees in which each gives a prediction. Finally, we collect the outputs from all the trees and consider the mean, median, or mode of this collection as the prediction of this forest. In the decision tree algorithm, there is only one tree trained on the entire data to make a prediction. However, as seen in Figure 2(B), the XGboost model is the slowest, followed by Random forest, and the decision tree is the fastest model with the lowest latency. Meanwhile, Random Forest has the largest file size, followed by XGBoost, and Decision Tree has the smallest file size, as shown in Figure 2(A).

Comparison Among Implemented Languages: It can be observed in Figure 2(A) that the maximum size of the model that can fit in a board differs based on the language in which the firmware is created. In our case, the maximum size of the model we can use for firmware made using C/C++ is 4.5MB, which was the size of the largest model used. Whereas the maximum size of the model we can use for firmware made using Micropython is nearly 600kB. Moreover, we have also observed that if a model under 600kB is complex, it causes a Bytecode Overflow while making the firmware. Hence, complication is also a factor we must consider when implementing a machine learning model into a micro-controller unit. In our case, the largest machine learning model we had implemented using Micropython was 588kB.

It can be observed in Figure 2(B) that implementing machine learning models in C/C++ is much faster than implementing machine learning algorithms in Micropython. The maximum latency for the C/C++ implementation was 1615 μ s, observed while using the XGBoost algorithm. The minimum latency for the C/C++ implementation was 41 μ s, observed while using the Decision Tree algorithm. Meanwhile, the maximum latency for the Micropython implementation was 5267 μ s, observed while using the XGBoost algorithm. In contrast, the minimum latency for the Micropython implementation was 184 μ s, which was observed while using the Decision Tree algorithm.

We can fit larger models in C/C++ implementation, and hence we can gain more accuracy from the C/C++ implementation. It can be observed from Figure 2(C) that the maximum accuracy in C/C++ implementation was obtained when using XGBoost, which is around 75.2%. The highest accuracy of our Micropython implementation using XGBoost was, however, 72.3%.

In Figure 3, we summarize the Decision Tree, Random Forest and XGBoost machine learning models as per the following metrics: Accuracy, Latency, File Size and Percentage of dataset used for training. We have shown the comparison for only the C/C++ language since it gives us higher accuracy. Hence, we would be finally using a C/C++ implementation.

Real-time Validation: We tested the XGBoost implementation of the system in C/C++ in an indoor environment and found 60% accuracy. The testing was done in early October, 2023, when there is a higher rainfall probability. Rainfall causes the AQI to decrease

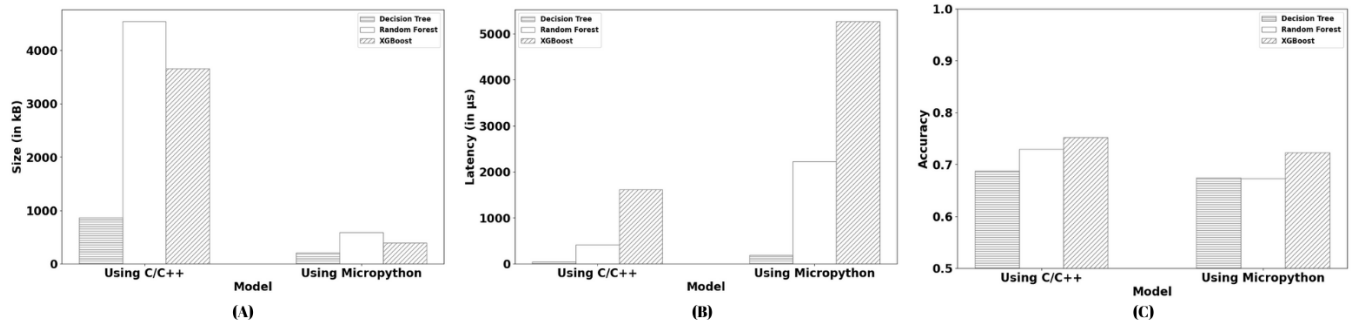


Figure 2: (A):Comparison of the sizes of models implemented in C and python, in kB. (B):Comparison of the latencies of models implemented in C and python, in μs . (C): Comparison of the accuracies of models implemented in C and python.

since there are fewer dust particles. Therefore, the model mainly predicted AQI 2, which means the air quality was satisfactory, though it may cause minor breathing discomfort to sensitive people. Since the device's location is in an industrial city, reaching AQI 1 is quite difficult.

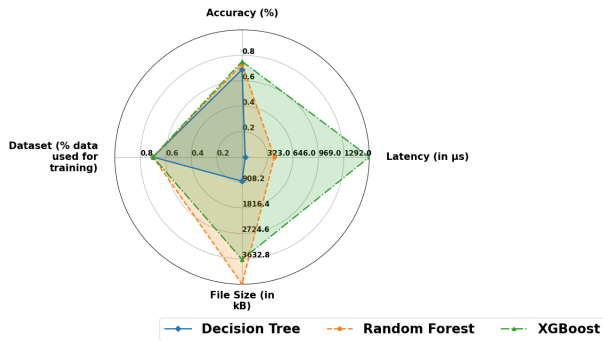


Figure 3: Performance trade-off diagram of the proposed device through the comparison of the accuracy, latency, file size and percentage of dataset for training, for the Decision Tree, Random Forest and XGBoost algorithms implemented in C/C++.

6 CONCLUSION & FUTURE WORK

In this article, we discussed a system that uses TinyML to predict the AQI of the user's real-time location. The board being used is constraint-limited, and the only sensor we use for AQI prediction is a humidity temperature sensor. Web crawling has been incorporated to gather meteorological and temporal data. Despite the flash memory space constraint, we have been successful in achieving 75.2% accuracy using the XGBoost algorithm implemented in C/C++. We have added a real-time validation system via a dust particle sensor that is used to infer the real AQI.

This system can be further improved if we employ a more robust real-time validation system that includes several more sensors to provide a better sense of the real-time AQI. The system model can also undergo further improvement by employing automated on-board training instead of training in the cloud. Another area

to be improved upon is the detection of pollution-causing events. Currently, we can only predict the outdoor AQI. The system can be improved to predict indoor AQI along with detection of events like fire, smoking, cooking without proper ventilation.

Such prototypes may be used to develop commercial-grade Air Quality Monitoring Systems using secondary sensing parameters without explicitly using a dust particle sensor, making it a low-cost solution. The zero-cloud processing of sensed data may lead to effective personalized sensing without compromising one's privacy.

ACKNOWLEDGMENTS

This publication is an outcome of the R&D work undertaken in the Project IntAirSense funded by Department of Science & Technology (DST), West Bengal, India for funding our research work in parts (Grant No. 228(Sanc.)/ST/P/S&T/6G-9/2018).

REFERENCES

- [1] Pedro Andrade, Ivanovitch Silva, Marianne Silva, Thommas Flores, Jordão Casiano, and Daniel G. Costa. 2022. A TinyML Soft-Sensor Approach for Low-Cost Detection and Monitoring of Vehicular Emissions. *Sensors* 22, 10 (May 2022), 3838.
- [2] Mattia Antonini, Miguel Pincheira, Massimo Vecchio, and Fabio Antonelli. 2023. An Adaptable and Unsupervised TinyML Anomaly Detection System for Extreme Industrial Environments. *Sensors* 23, 4 (Feb. 2023), 2344.
- [3] Prithviraj Pramanik, Prasenjit Karmakar, Praveen Kumar Sharma, Soumyajit Chatterjee, Abhijit Roy, Santanu Mandal, Subrata Nandi, Sandip Chakraborty, Mousumi Saha, and Sujoy Saha. 2023. *AQuaMoHo*: Localized Low-cost Outdoor Air Quality Sensing over a Thermo-hygrometer. *ACM Transactions on Sensor Networks* 19, 3 (March 2023), 1–30.
- [4] Partha Pratim Ray. 2022. A review on TinyML: State-of-the-art and prospects. *Journal of King Saud University - Computer and Information Sciences* 34, 4 (April 2022), 1595–1623.
- [5] Praveen Kumar Sharma, Prasenjit Karmakar, Soumyajit Chatterjee, Abhijit Roy, Santanu Mandal, Sandip Chakraborty, Subrata Nandi, and Sujoy Saha. 2021. Can I go for a roof walk today?. In *Proceedings of the 8th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation*. ACM.
- [6] Vasileios Tsoukas, Eleni Boumpa, Georgios Giannakas, and Athanasios Kakarountas. 2021. A Review of Machine Learning and TinyML in Healthcare. In *25th Pan-Hellenic Conference on Informatics*. ACM.
- [7] Vasileios Tsoukas, Anargyros Gkogkidis, and Athanasios Kakarountas. 2022. A TinyML-based System For Smart Agriculture. In *Proceedings of the 26th Pan-Hellenic Conference on Informatics*. ACM.